ARCHITECTURE AUDIT SUMMARY

Argus: Current System vs. Proposed North-Star Architecture

Concise synthesis for Fan and Argus developers, distilled from the completed repository-wide audit.

> **Executive verdict:** Fan's model is a **better north-star** / **metacognitive control plane** for Argus, but it should be layered on top of the current conservative execution spine — **not** treated as a wholesale replacement, unconstrained goal rewriting, or autonomous self-modification system.

What Argus is today

A persistent, verticalized autonomous research/work runtime with Manager, Planner, Engineer, and Reviewer roles; backlog/DAG scheduling; per-stage checklists; durable project state; evidence-gated completion; budget guards; project skills/wiki; and explicit human questions for operator-only blockers.

What Fan's model adds

A cleaner top-level control layer: typed goal refinement, precise-vs-semantic clause handling, dynamic role composition under budget, explicit hierarchical memory, typed HITL triggers, and governed learning/self-improvement proposals.

Audited commit	`eb57a6aa164421cb7ffc09505a720977283b4798`
Remote status	Public/remote HEAD could not be certified; unauthenticated access returned 404/credential prompt, so the audit used an installed local checkout snapshot.
Testing caveat	Full suite did not collect in available environments due missing optional/dev dependencies; an architecture-relevant bounded subset passed.
Security note	This summary intentionally omits credentials and sensitive local filesystem paths.

1. Accurate current mechanisms

Control spine

- **Manager** is the front door and stage authority: it interprets operator intent, selects a vertical/stage, and writes stage transitions fail-closed.
- **Planner** decomposes work into tasks/DAG nodes with keys, dependencies, and cost-aware bounded planning.
- **Engineer** executes bounded work in fresh provider sessions using tools, files, experiments, and shared checkpoint continuity.
- **Reviewer** gates completion with structured outcomes: `done`, `continue`, or `blocked`.

Persistence and governance

- LifeSupervisor runs durable backlog missions with status, budget, dependency metadata, pending questions, and context references.
- Raw low-level provider context is deliberately reset between rounds/items to reduce contamination; continuity is carried by project state and checkpoint files.
- Project skills/wiki evolution exists, but protected floors and source-promotion gates constrain self-modification.
- Budgeting is real, mostly via provider spend/cost guards rather than a mature token-composition optimizer.

Short accurate description

Argus is a persistent, evidence-gated, budget-aware research/work runtime. It already supports long-horizon pursuit, role-separated planning/execution/review, durable memory, project-level learning artifacts, and human-in-the-loop blockers. It is not yet a fully generic continuous goal-refinement engine or autonomous self-PR generator.

Implemented center of gravity

Area	Current state
Long-horizon execution	central LifeSupervisor + backlog/DAG + durable project state.
Role split	central Manager / Planner / Engineer / Reviewer, with Reviewer as completion authority.
Evidence gates	central Checklist/evidence/contracts are core to completion decisions.
Learning	partial Project skills/wiki and gated promotion exist; several activation paths are transitional or default-off.
Budget awareness	partial Cost/budget controls are stronger than explicit token-aware role composition.

2. Main overstatements and not-yet-implemented pieces

Claim in Fan's model	Audit classification	Precise correction
General-purpose engine for any non-trivial work	partial	Broader than one benchmark and domain-agnostic in plumbing, but still verticalized and not fully generic for “any work.”
Continuous goal/subgoal refinement whenever necessary	partial	Replanning, rollback, `step_back`, and Dynamic Plan exist, but goal text is not freely or continuously rewritten; Dynamic Plan is default-off.
Explicit precise-vs-semantic goal mode	aspirational	Hard criteria and checklist prose exist; no first-class typed goal representation was found.
Token-budget-aware harness composition	weak partial	There is spend accounting and bounded-DAG cost awareness, but not a central optimizer choosing role harnesses by token budget.
Dynamic arbitrary role/subagent workflow	partial	Some dynamic team/background-task behavior exists, but the main loop remains mostly fixed.
Hierarchical memory with volatile low-level reset	partial	Persistent project state and fresh sessions are real, but the memory hierarchy is not yet a formal typed model.
Autonomous self-PRs to improve Argus	absent	No implemented GitHub PR path was found. Source writeback is local, opt-in, and default-off.

Key correction: current Argus intentionally favors conservative execution: evidence gates, protected skill floors, local/default-off writeback, and human questions for true blockers. Fan's model should preserve those guardrails rather than weaken them.

What should not be claimed as already implemented

- A fully generic “any work” reasoning engine.
- A central semantic-vs-precise goal-refinement module.
- Always-on continuous dynamic replanning of goals.
- A token-budget optimizer for choosing arbitrary role harnesses.
- Autonomous PR creation/merge or unconstrained self-modification.
- Fully activated, ungated learning that safely edits future system behavior.

3. Is Fan's model better?

> **Yes — as a roadmap.** Fan's proposal improves Argus by making the currently fragmented meta-control ideas explicit: goal contracts, role planning, memory layers, and typed HITL. **No — if interpreted as unconstrained autonomy.**

Adopt as north-star

- **GoalContract:** original request, semantic intent, exact constraints, excluded outcomes, success tests, ambiguities, and revision history.
- **Precise/semantic clauses:** exact constraints verified mechanically where possible; semantic clauses verified by rubric and evidence.
- **RolePlan:** choose the simplest safe harness, escalating only when uncertainty/risk/budget justify it.
- **Hierarchical memory:** immutable request, Manager-certified goal contract, Planner DAG, Reviewer-certified evidence, volatile scratch.

Guardrails required

- Manager owns changes to semantic intent and hard constraints.
- Reviewer verifies that refinements preserve non-negotiables.
- Every goal revision must cite evidence and retain an immutable original-request anchor.
- Self-improvement outputs should be proposals/branches/tests only until human approval.
- No mechanism may weaken protected policies, secret handling, budget caps, sandboxing, or HITL thresholds without explicit review.

Typed artifacts Fan's model should introduce

Artifact	Purpose	Owner / gate
`GoalContract`	Stable goal truth: exact constraints + semantic intent + success tests.	Manager amends; Reviewer audits preservation.
`RolePlan`	Budget/risk-aware selection of Planner, Engineer, Reviewer, parallel teams, or adversarial review.	Manager/Planner propose; budget/risk caps constrain.
Fact/evidence ledger	Reviewer-certified facts with provenance and retractions.	Reviewer-certified, append-oriented.
HITL trigger schema	Mandatory human questions for ambiguity, irreversible actions, external writes, protected changes, safety/privacy, or budget expansion.	System-enforced; not optional prompt style.

4. Adopt / modify / reject priorities

P0 — Adopt or modify now

1. **Typed GoalContract.** Make original request, exact constraints, semantic intent, success tests, exclusions, ambiguity triggers, and revisions first-class.
2. **Precise-vs-semantic clause typing.** Replace prompt-only hard criteria with structured fields used by Planner and Reviewer.
3. **Memory hierarchy hardening.** Separate free-form checkpoint from structured goal/fact/evidence ledgers; alarm on stale/corrupt memory.
4. **Typed HITL thresholds.** Require human input for external writes, protected self-modification, budget expansion, ambiguity, privacy/safety, and irreversible actions.

P1 — Adopt carefully

1. **Dynamic role/harness composition.** Add `RolePlan` with expected cost, risk tier, verification depth, and minimum independent review.
2. **Guarded Dynamic Plan expansion.** Consider default-on only for long-running open-ended campaigns after GoalContract preservation checks exist.
3. **Task-local tactics distinct from reusable skills.** Promote only after repeated verified utility.

P2 — Keep gated / default-off

1. **Self-improvement PR generation.** Treat as proposal/branch/test generation requiring human approval before public PRs or protected-policy changes.
2. **Source promotion/autocommit.** Current default-off posture is correct; add review bundles before enabling more automation.

Reject or defer

- **Unconstrained continuous goal rewriting.** Too much goal-drift risk.
- **Self-verification as sufficient completion.** Keep independent Reviewer and deterministic checks; add adversarial/external review for high-stakes claims.

5. Risks, caveats, and bottom line

Primary risks to manage

- **Goal drift:** refinements can weaken the original objective unless anchored and versioned.
- **Circular self-verification:** Reviewer independence is role-level, not perfect epistemic independence.
- **Weak token planning:** spend guards exist, but context/token role composition remains underdeveloped.
- **Self-modification governance:** runtime skills/overlays can affect future behavior; provenance and protected floors must remain strong.

Operational caveats from the audit

- **Remote HEAD could not be certified.** Public unauthenticated access failed, so the audited commit is a local snapshot rather than authoritative upstream HEAD.
- **Full test suite did not run cleanly.** Collection failed in available environments because optional/dev dependencies were missing.
- **Bounded evidence passed.** Representative core loop, supervisor, dynamic plan, budget/HITL, session resume, and stage-decider tests passed.
- **Line-level audit, not runtime-host certification.** No private host access was used.

Skill / Tactic / Finding / Guardrail distinction

Type	Meaning	Promotion rule
Skill	Reusable procedure or playbook.	Promote after repeated verified utility.
Tactic	Task-local method that may die with the project.	Keep local unless repeatedly useful.
Finding	Factual/project memory with provenance.	Store in evidence/fact ledger with retraction support.
Guardrail	Protected policy or safety boundary.	Hard to weaken; require explicit human review.

Bottom line: Argus should keep its current conservative execution spine and add Fan's model as a typed metacognitive control plane. That means adopting GoalContract, RolePlan, hierarchical memory, and typed HITL — while rejecting unconstrained goal rewriting, self-verification as sufficient proof, and autonomous self-modification without human-governed review.

Evidence basis: completed repository-wide source audit at commit `eb57a6aa164421cb7ffc09505a720977283b4798`; representative bounded tests passed; remote/public HEAD was not certified.